

AI Engineering

Building Applications with Foundation Models

Chip Huyen

O'REILLY®

[OceanofPDF.com](https://oceanofpdf.com)

be obvious what caused this gradual drop. You need investigations to understand the root cause.

Underlying model changes

When using a model through an API, it's possible that the API remains unchanged while the underlying model is updated. As mentioned in [Chapter 4](#), model providers might not always disclose these updates, leaving it to you to detect any changes. Different versions of the same API can have a significant impact on performance. For instance, [Chen et al. \(2023\)](#) observed notable differences in benchmark scores between the March 2023 and June 2023 versions of GPT-4 and GPT-3.5. Likewise, Voiceflow reported a [10% performance drop](#) when switching from the older GPT-3.5-turbo-0301 to the newer GPT-3.5-turbo-1106.

AI Pipeline Orchestration

An AI application can get fairly complex, consisting of multiple models, retrieving data from many databases, and having access to a wide range of tools. An orchestrator helps you specify how these different components work together to create an end-to-end pipeline. It ensures that data flows seamlessly between components. At a high level, an orchestrator operates in two steps, components definition and chaining:

Components definition

You need to tell the orchestrator what components your system uses, including different models, external data sources for retrieval, and tools that your system can use. A model gateway can make it easier to add a model.⁶ You can also tell the orchestrator if you use any tools for evaluation and monitoring.

Chaining

Chaining is basically function composition: it combines different functions (components) together. In chaining (pipelining), you tell the orchestrator the steps your system takes from receiving the user query until completing the task. Here's an example of the steps:

1. Process the raw query.
2. Retrieve the relevant data based on the processed query.
3. Combine the original query and the retrieved data to create a prompt in the format expected by the model.
4. The model generates a response based on the prompt.
5. Evaluate the response.
6. If the response is considered good, return it to the user. If not, route the query to a human operator.

The orchestrator is responsible for passing data between components. It should provide toolings that help ensure that the output from the current

step is in the format expected by the next step. Ideally, it should notify you when this data flow is disrupted due to errors such as component failures or data mismatch failures.

WARNING

An AI pipeline orchestrator is different from a general workflow orchestrator, like Airflow or Metaflow.

When designing the pipeline for an application with strict latency requirements, try to do as much in parallel as possible. For example, if you have a routing component (deciding where to send a query) and a PII removal component, both can be done at the same time.

There are many AI orchestration tools, including [LangChain](#), [LlamaIndex](#), [Flowise](#), [Langflow](#), and [Haystack](#). Because retrieval and tool use are common application patterns, many RAG and agent frameworks are also orchestration tools.

While it's tempting to jump straight to an orchestration tool when starting a project, *you might want to start building your application without one first*. Any external tool brings additional complexity. An orchestrator can abstract away critical details of how your system works, making it hard to understand and debug your system.

As you advance to the later stages of your application development process, you might decide that an orchestrator can make your life easier. Here are three aspects to keep in mind when evaluating orchestrators:

Integration and extensibility

Evaluate whether the orchestrator supports the components you're already using or might adopt in the future. For example, if you want to use a Llama model, check if the orchestrator supports that. Given how many models, databases, and frameworks there are, it's impossible for an orchestrator to support everything. Therefore, you'll also need to consider an orchestrator's extensibility. If it doesn't support a specific component, how hard is it to change that?

Support for complex pipelines

As your applications grow in complexity, you might need to manage intricate pipelines involving multiple steps and conditional logic. An orchestrator that supports advanced features like branching, parallel processing, and error handling will help you manage these complexities efficiently.

Ease of use, performance, and scalability

Consider the user-friendliness of the orchestrator. Look for intuitive APIs, comprehensive documentation, and strong community support, as these can significantly reduce the learning curve for you and your

team. Avoid orchestrators that initiate hidden API calls or introduce latency to your applications. Additionally, ensure that the orchestrator can scale effectively as the number of applications, developers, and traffic grows.

User Feedback

User feedback has always played a critical role in software applications in two key ways: evaluating the application's performance and informing its development. However, in AI applications, user feedback takes on an even more significant role. User feedback is proprietary data, and data is a competitive advantage. A well-designed user feedback system is necessary to create the data flywheel discussed in [Chapter 8](#).⁷

User feedback can be used not only to personalize models for individual users but also to train future iterations of the models. As data becomes increasingly scarce, proprietary data is more valuable than ever. A product that launches quickly and attracts users early can gather data to continually improve models, making it difficult for competitors to catch up.

It's important to remember that user feedback is user data. Leveraging user feedback requires the same cautions needed when leveraging any data. User privacy should be respected. Users have the right to know how their data is being used.